Reporte

Nombre: Andrés Daniel Pérez Ordaz Jonathan Esquivel Anaya Patricia Lizeth Ramos Ortiz	**Matrícula:** 2163200 2160886 2161739
Nombre del curso: Componentes de lógica de negocio	**Nombre del profesor:** Miguel Ángel Gómez Marroquín
Modulo Netacad: 3	**Actividad:** Actividad Unidad 3: Diseño arquitectural del sistema con IA
Fecha: 17/10/2026	
Bibliografia (Formato APA) Evans, E. (2017). Domain-Driven Design: Tackling Complexity in the Heart of Software (Trad. al español). Addison-Wesley / Pearson Educación. Freund, J., & Rücker, B. (2020). BPMN 2.0: Manual de referencia y guía práctica. Marcombo.	

Objetivo:

Convertir el análisis de negocio de la Unidad 2 de CleanPro (BRD, historias de usuario, BPMN y glosario) en una propuesta de arquitectura de software que separe dominio, aplicación e infraestructura, justificando cada decisión con evidencia del caso.

Procedimiento:

Alcance

Este informe documenta el diseño arquitectural del sistema de CleanPro, empresa de servicios de limpieza residencial y empresarial, como continuación del análisis de negocio de la Unidad 2 (BRD, historias de usuario con criterios Gherkin, BPMN y glosario). Se trabaja con la misma empresa y el mismo caso.

Proceso principal de diseño detallado. Va desde la aceptación de la cotización hasta el cierre del servicio para facturación. Incluye la programación tentativa, la confirmación de cuadrilla el día previo, la ejecución en sitio, las incidencias y los extras.

Casos de uso seleccionados.

- CU-01 Aceptar cotización.
- CU-02 Confirmar cuadrilla el día previo.

- CU-03 Reportar ausencia y reasignar personal.
- CU-04 Registrar extra solicitado en sitio.
- CU-05 Cerrar servicio para facturación.

Los cinco aparecen en las historias de usuario y en el BPMN (ver tablas siguientes). Incluyen flujos normales y flujos de excepción o rechazo, como cuadrilla no disponible, ausencia sin reemplazo, extra fuera de catálogo y cierre bloqueado por pendientes.

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
CU-01	Aceptar cotización	Cliente	BR-01	La cotización pasa a Aceptada y se crea una programación tentativa vinculada a su folio, sin cuadrilla definitiva.
CU-02	Confirmar cuadrilla el día previo	Coordinador	BR-02, RNF-02	La cuadrilla queda confirmada solo si el coordinador valida su disponibilidad real; si no, no se considera confirmada.
CU-03	Reportar ausencia y reasignar personal	Supervisor (reporta) y Administración (reasigna)	BR-03, RNF-01, RNF-03	Se crea una incidencia vinculada al servicio, Administración recibe aviso inmediato y la reasignación conserva al empleado original y al sustituto. Sin reemplazo, el servicio queda en riesgo.
CU-04	Registrar extra solicitado en sitio	Supervisor (y Coordinador si el extra está fuera del catálogo)	BR-04, RNF-02, RNF-03	Un extra del catálogo se acepta directamente; uno fuera del catálogo queda pendiente de confirmación del coordinador.
CU-05	Cerrar servicio para facturación	Administración	BR-07, RNF-02	El servicio se cierra si no tiene incidencias ni extras pendientes de documentar; si los tiene, se bloquea y el sistema indica cuál falta.

Comprobación contra la Unidad 2.

Los cinco casos aparecen en las historias de usuario y en el BPMN, por lo que no requieren justificación como corrección de alcance. Cada uno incluye un flujo de excepción o rechazo:

ID	Evidencia en la Unidad 2	Flujo de excepción o rechazo
CU-01	US-03 (con US-01 y US-02 como precondición); BPMN: «Revisar y aceptar cotización» y «¿Aceptada?»	Cotización que no está Vigente: se rechaza la aceptación (COTIZACION_NO_VIGENTE).
CU-02	US-04 y US-05; BPMN: «Confirmar cuadrilla (día previo)» y «¿Cuadrilla disponible?»	Cuadrilla no disponible: CUADRILLA_NO_DISPONIBLE. La alternativa posterior no está definida en la U2 (H-05).
CU-03	US-06 y US-07; BPMN: «Reasignar personal», «¿Reemplazo encontrado?» y «Notificar riesgo al cliente»	Sin reemplazo disponible: el servicio pasa a EnRiesgo y se sugiere contactar al cliente.

CU-04	US-10; BPMN: «Registrar incidencia / extra / ajuste»	Extra fuera del catálogo: queda Pendiente de confirmación del coordinador.
CU-05	US-17; BPMN: «Verificar incidencias y extras pendientes» y «¿Pendientes por documentar?»	Incidencias o extras pendientes: CIERRE_BLOQUEADO con el detalle de lo que falta.

Quedan fuera de los cinco casos, aunque dentro del alcance del BRD: emitir cotización (US-01, US-02), ajuste por inmueble mayor (US-11), acceso negado (US-13), reprogramación (US-14, US-15), cancelación (US-16) y consultas de panel e historial (US-18, US-19). Se diseñan en el dominio, pero no se programan en el código base (ver Alcance del código base).

Qué cubre este documento.

- Comparación de patrones de lógica de negocio.
- Mapa de dominio.
- Diseño de Service Layer.
- Repository y Unit of Work.
- Separación de Commands y Queries (CQRS básico).
- Revisión con SOLID.
- Registro de decisiones (ADR).

Se acompaña de diagramas de clases y de secuencia, una matriz de trazabilidad, una bitácora de uso de IA y un código base con pruebas.

Supuestos y límites. No se inventaron valores ni políticas que la Unidad 2 deja pendientes. El tiempo de cortesía, el cobro por acceso negado, la penalización por reprogramación tardía, el criterio de precio «significativo» y el catálogo de extras quedan como políticas configurables sin valor asignado (ADR-003). Los demás datos que la U2 no define (folio único, tarifario, moneda, unidades de superficie, definición de Empleado) se registran en la sección Hallazgos y pendientes sobre la Unidad 2 (H-01 a H-08). Los valores de las pruebas del código son ilustrativos.

Nomenclatura de reglas. El BRD no numera sus reglas ni sus requisitos no funcionales. La numeración usada en todo el informe, la matriz de trazabilidad y la bitácora es del equipo y sigue el orden del texto del BRD (H-07):

ID	Regla o requisito del BRD	Orden en el BRD
BR-01	Una cotización no compromete cuadrilla; solo la aceptación del cliente genera una programación tentativa.	Reglas de negocio, 1.ª
BR-02	La asignación final de personal se confirma un día antes, según disponibilidad real.	Reglas de negocio, 2.ª
BR-03	Toda ausencia se reporta al detectarse y queda como incidencia vinculada al servicio.	Reglas de negocio, 3.ª

BR-04	Extra del catálogo: lo acepta el supervisor; si modifica el precio de forma significativa requiere al coordinador.	Reglas de negocio, 4.ª
BR-05	Inmueble mayor al reportado: el coordinador recalcula y el cliente acepta el ajuste antes de continuar.	Reglas de negocio, 5.ª
BR-06	Reprogramación de última hora: la solicita el cliente y la decide Administración.	Reglas de negocio, 6.ª
BR-07	Un servicio no se cierra para facturación con incidencias o extras sin documentar.	Reglas de negocio, 7.ª
BR-08	Modificar o cancelar un servicio programado corresponde al coordinador; Administración queda notificada.	Reglas de negocio, 8.ª
RNF-01	Los cambios de estado de un servicio son visibles para Administración sin demoras significativas.	Requisitos no funcionales, 1.º
RNF-02	Las funciones sensibles (ajustes, reprogramaciones, cierre) están limitadas por rol.	Requisitos no funcionales, 2.º
RNF-03	Toda modificación relevante conserva quién, cuándo y por qué.	Requisitos no funcionales, 3.º
RNF-04	Los valores no confirmados (tiempo de cortesía, cobro por acceso negado, penalización por reprogramación tardía) son configurables.	Requisitos no funcionales, 5.º

Alcance del código base. El código base implementa CU-01 a CU-05 con sus 8 Commands, la consulta de disponibilidad (US-05) y 35 pruebas. No implementa: emitir cotización como caso de uso (solo existe Cotizacion.emitir), CalculadoraDeEstimacion, ajuste por inmueble mayor, acceso negado, reprogramación, cancelación, panel operativo e historial (US-11 a US-16, US-18 y US-19). Los elementos del mapa de dominio que responden a esas historias están marcados como diseño sin implementar. Los repositorios del código son en memoria; el ORM es la propuesta de producción.

Comparación de patrones

Criterio del caso	Transaction Script	Table Module	Domain Model	Evidencia U2
Complejidad de reglas	Un procedimiento por historia. Sirve para pasos simples, pero las reglas que dependen de rol, estado y configuración (BR-04, 05, 06, 07) se repetirían en varios procedimientos.	Una clase por tabla (Cotización, Servicio, Incidencia, Extra). Las reglas se reparten por tabla, pero BR-07 cruza tres.	Cada regla vive en el objeto que la protege (la de cierre, en Servicio), con un único punto de verdad.	BRD: Reglas de negocio BR-01 a BR-08; US-10, US-11, US-15, US-17
Cambios esperados	Cambiar el tiempo de cortesía o la penalización obliga a editar cada script donde se use.	El cambio queda en el módulo de la tabla, pero las políticas no tienen un objeto que las represente.	Las políticas configurables pueden encapsularse sin tocar el flujo principal.	BRD: RNF-04, Restricciones y supuestos; riesgo de que cada supervisor aplique las políticas distinto; US-13
Interacción entre conceptos	Cada script debe orquestar a mano Cotización, Servicio, Cuadrilla, Incidencia y Extra, y esa coordinación se duplica.	Los módulos no se conocen entre sí; haría falta otra capa que los coordine.	Servicio agrupa cotización aceptada, cuadrilla, registros, incidencias y extras; la relación es explícita.	Glosario: Servicio, Incidencia, Reasignación de personal; US-06 y US-07; US-19; BPMN con 5 carriles
Estados e invariantes	El estado es un valor en tabla que cada script debe validar; hay riesgo de transiciones inconsistentes.	El estado es una columna validada por módulo; las invariantes entre tablas son difíciles de garantizar.	Las transiciones e invariantes viven dentro de la entidad, y no hay otro camino para llegar a un estado inválido.	Glosario: Vigente/Aceptada/Rechazada, Servicio en riesgo, Servicio no realizado, Cierre para facturación; US-08, US-13, US-17; eventos de fin del BPMN
Criterio adicional: lenguaje ubicuo y trazabilidad	Los nombres tienden a ser verbos de procedimiento y reflejan poco el glosario.	Los nombres siguen a las tablas, no a los conceptos del negocio.	Clases y métodos reutilizan el glosario (Servicio, Incidencia, Cierre para facturación) y las modificaciones	Glosario (cita a Evans, DDD); RNF-03

			pueden registrar quién, cuándo y por qué.	

Ventajas, riesgos y consecuencias de cada alternativa

Transaction Script.

- **Ventajas:** es el más simple y rápido de construir. Encaja bien con pasos sin reglas complejas, como consultar la disponibilidad (US-05), el panel (US-18) o el historial (US-19).
- **Riesgo:** duplicación de reglas. Cualquier procedimiento que decida si un servicio es facturable tendría que repetir la verificación de BR-07, que en el BPMN es el gateway “¿Pendientes por documentar?”.
- **Consecuencia para CleanPro:** cuando el negocio valide las políticas pendientes (cortesía, penalización, material especial), habría que revisar muchos scripts. El riesgo que el BRD ya identifica, que cada supervisor aplique las políticas de forma distinta, pasaría al código.

Table Module.

- **Ventajas:** se adapta bien a datos tabulares y a reportes.
- **Riesgo:** las reglas quedan repartidas por tabla, mientras que BR-03 y BR-07 cruzan Servicio, Incidencia y Extra. Alguien tendría que coordinar los módulos, y eso terminaría siendo Transaction Script con otro nombre.
- **Consecuencia para CleanPro:** las invariantes del cierre para facturación quedarían sin un dueño claro.

Domain Model.

- **Ventajas:** protege las invariantes en un solo lugar, reutiliza el lenguaje del glosario y permite que las políticas pendientes sean configurables (RNF-04).
- **Riesgos y costos:** más esfuerzo inicial de modelado y más trabajo para mapear a persistencia. Hay riesgo de sobre-modelar, convirtiendo cada tabla en entidad. Además, exige que el equipo domine el patrón.

Decisión

El patrón principal es Domain Model para el flujo que cambia el estado de un servicio: aceptación de la cotización, programación tentativa, confirmación de cuadrilla, ejecución, incidencias, extras y ajustes, cierre para facturación, ausencias y reasignación, reprogramación y cancelación. Esto cubre los cinco casos de uso de la Etapa 1.

Se propone una combinación delimitada. Transaction Script se usa solo para las operaciones de lectura que no cambian estado ni protegen invariantes: disponibilidad

por día (US-05), panel operativo (US-18) e historial de servicio (US-19). Table Module se descarta porque sus reglas por tabla no pueden garantizar invariantes que cruzan Servicio, Incidencia y Extra.

Problema concreto que ayuda a resolver. Dos problemas observados en el caso son los errores de facturación y la poca evidencia de incidencias y extras. La regla que los ataca es BR-07 (US-17): un servicio no puede cerrarse para facturación con pendientes. Con Domain Model esa regla se aplica en un solo lugar, y ninguna pantalla ni proceso puede saltársela. Lo mismo ocurre con otras invariantes:

- no dar por confirmada una cuadrilla antes del día previo (BR-02, US-04);
- no registrar el término antes del inicio (US-08);
- no continuar con un ajuste sin aceptación del cliente (BR-05, US-11);
- no registrar un acceso negado antes del tiempo de cortesía (US-13).

Costo que se acepta. El sistema no es grande: son 19 historias y 5 actores, y el BRD deja fuera la optimización de rutas, la integración contable y la nómina. La justificación no es el tamaño, sino los estados y las invariantes. El costo es modelar con más cuidado y acotar el Domain Model al flujo central, sin extenderlo a todo el sistema. Se acepta más esfuerzo inicial y una capa de mapeo a persistencia, a cambio de no repartir BR-07 y las demás invariantes en varios lugares.

Mapa de dominio

Base del diseño

El patrón elegido es Domain Model para el flujo que cambia el estado de un servicio, y los casos de uso son CU-01 a CU-05: aceptar cotización, confirmar cuadrilla, reportar ausencia y reasignar, registrar extra en sitio y cerrar para facturación. Las consultas (disponibilidad, panel, historial) quedan como Transaction Script.

Límites del dominio

Monolito modular con dos contextos y un componente de soporte. Los contextos se comunican por identificadores (Folio, CuadrillaId, EmpleadoId) y por Domain Events. La única excepción transaccional es la aceptación de la cotización (excepción documentada en ADR-002).

Contexto	Contiene	Responde a
Cotización	Cotización (con DatosDelInmueble, Frecuencia, extras cotizados, Estimación)	US-01, US-02, US-03; BR-01; glosario: Cotización
Operación del servicio	Servicio (con Incidencias, Extras, RegistroDeTiempo, AsignacionDePersonal; en diseño y sin implementar: AjusteDePrecioYTiempo, SolicitudDeReprogramacion), Cuadrilla, Empleado	US-04 a US-17; BR-02 a BR-08; todo el BPMN desde "Confirmar cuadrilla" hasta "Verificar incidencias y extras pendientes"
Soporte: ConfiguracionDePoliticas	Parámetros revisables por Administración: tiempo de cortesía, catálogo de extras, criterio de "precio significativo", política de cobro por acceso negado y penalización por reprogramación tardía	RNF-04; BRD: valores pendientes de validar
Fuera del dominio: Facturación	Recibe el evento ServicioCerradoParaFacturacion. No se modela Factura	BRD: alcance excluido; BPMN: "Facturar servicio"

Lenguaje ubicuo

Un término significa lo mismo en BRD, historias, BPMN, UML, código y ADR. En el código se usan los nombres del glosario en español.

Término	Uso en el modelo	Evidencia
Cotización	Aggregate con estados Vigente, Aceptada y Rechazada	Glosario; US-01; US-03
Folio	Identificador único asignado al emitir la cotización. El Servicio que nace de ella conserva el mismo Folio (supuesto a validar, H-01)	US-01; US-03; US-06; US-08
Servicio	Aggregate que agrupa, vinculados por el Folio de la cotización aceptada, la cuadrilla confirmada, los registros de inicio y término, incidencias y extras. La Cotización es otro Aggregate (ADR-002)	Glosario: Servicio
Programación tentativa	Estado inicial del Servicio (Programado): fecha comprometida sin cuadrilla confirmada	Glosario; BR-01; US-03
Confirmación de cuadrilla	Acto del coordinador, un día antes, que valida disponibilidad real	Glosario; BR-02; US-04
Cuadrilla / Empleado	Grupo de personal operativo y sus integrantes	Glosario: Cuadrilla; US-06; US-07; Empleado no está definido en la U2 (H-04)
Incidencia	Entidad del Servicio con tipo (ausencia, acceso negado, material especial, daño reportado), descripción, hora y estado de revisión	Glosario; US-09
Extra (en sitio)	Entidad del Servicio: solicitud del cliente en sitio no incluida en la cotización. Es el "Extra / servicio adicional" del glosario	Glosario; US-10; BR-04
Extra cotizado	Extra que forma parte de la cotización original (distinto del extra en sitio)	US-01; Glosario actualizado
Catálogo de extras	Lista predefinida que decide si el supervisor acepta directo	Glosario; US-10
Ajuste de precio y tiempo	Recálculo por inmueble mayor; requiere aceptación explícita del cliente	Glosario; BR-05; US-11
Servicio en riesgo / no realizado	Estados por ausencia sin reemplazo y por acceso negado	Glosario; US-07; US-13
Cierre para facturación	Estado que permite facturar; solo sin pendientes por documentar	Glosario; BR-07; US-17

Inventario del dominio

Entidades y Aggregates

Elemento	Tipo	Identidad	Por qué tiene identidad
Cotización	Aggregate root	Folio	Cambia de estado (Vigente, Aceptada, Rechazada) sin dejar de ser la misma
Servicio	Aggregate root	Folio	Ciclo de vida completo desde la programación tentativa hasta el cierre (US-19: historial)
Incidencia	Entidad del Servicio	Id local	Queda Pendiente de revisión y luego se atiende (US-09)
Extra	Entidad del Servicio	Id local	Tiene estado: aceptado o pendiente de confirmación (US-10)
AjusteDePrecioYTiempo	Entidad del Servicio	Id local	Queda pendiente de aceptación del cliente (US-11) Diseño: no está implementada en el código base.
SolicitudDeReprogramacion	Entidad del Servicio	Id local	Está pendiente hasta que Administración decide (US-14, US-15) Diseño: no está implementada en el código base.
Cuadrilla	Aggregate root (mínimo)	CuadrillaId	Se distingue aunque cambien sus integrantes
Empleado	Aggregate root (mínimo)	EmpleadoId	Se reasigna y se registra como original o sustituto (US-07)
ConfiguracionDePoliticas	Aggregate root (soporte)	Versión	Los valores pendientes son revisables y deben quedar versionados (RNF-03, RNF-04)

Cliente no se modela como Aggregate: la U2 no define comportamiento ni datos propios del cliente más allá de solicitar, aceptar y pedir reprogramaciones. Se referencia como ClienteId.

Value Objects

Value Object	Reglas que protege	Evidencia
Folio	Único; se asigna al emitir la cotización	US-01
DatosDelInmueble	Ubicación, superficie (mayor que 0) y tipo de inmueble, los tres obligatorios	US-01
Frecuencia	Valor de catálogo; obligatorio	US-01; glosario

Value Object	Reglas que protege	Evidencia
Estimacion	Duración, personal y precio calculados a partir de superficie, tipo, frecuencia y extras; el coordinador puede editarla antes de emitir	US-02
Dinero	Monto no negativo. La moneda no está definida en la U2	BRD
RegistroDeTiempo	Inicio y término con fecha y hora exactas; el término no es anterior al inicio	US-08
AsignacionDePersonal	Empleado original, sustituto, quién, cuándo y por qué	US-07; RNF-03
FechaDeServicio	Fecha y hora programadas	US-03; US-04
Motivo	Razón registrada de un ajuste, rechazo o cancelación	US-11; US-15; RNF-03

Domain Services

Solo se usan cuando la regla involucra más de un Aggregate o depende de configuración.

Domain Service	Responsabilidad	Por qué no vive en una entidad	Evidencia
CalculadoraDeEstimacion	Propone duración, personal y precio con superficie, tipo, frecuencia y extras	Usa un tarifario que no pertenece a una cotización. El tarifario no está definido en la U2 (H-06). Diseño: no está implementado en el código base ni en el diagrama de clases	US-02
AsignadorDeCuadrilla	Verifica que la cuadrilla propuesta esté libre en la fecha, a partir de los servicios ya confirmados	Cruza Servicio y Cuadrilla; la disponibilidad se deriva de los servicios confirmados	US-04; US-05; BR-02
PoliticaDeExtras	Decide si un extra está en el catálogo (aceptación directa) o requiere confirmación del coordinador	El catálogo es configuración; el criterio de precio "significativo" está sin definir	US-10; BR-04

Se descartó SelectorDeReemplazo: en la U2 es Administración quien elige al sustituto; el sistema solo muestra el personal disponible (consulta).

Domain Events

Evento	Lo emite	Le interesa a	Evidencia
CotizacionEmitida	Cotización	Cliente (revisión y aceptación)	US-01
CotizacionAceptada	Cotización	Operación: crea el Servicio	US-03; CU-01
CotizacionRechazada	Cotización	Historial	Glosario; BPMN
ServicioProgramado	Servicio	Coordinador	US-03
RecordatorioDeConfirmacionGenerado	Servicio (disparado a 24 h)	Coordinador	US-04
CuadrillaConfirmada	Servicio	Supervisor, Administración	CU-02; BR-02
AusenciaReportada	Servicio	Administración (aviso inmediato)	US-06; RNF-01
PersonalReasignado	Servicio	Administración	US-07
ServicioEnRiesgo	Servicio	Administración (sugerir contactar al cliente)	US-07
ServicioIniciado / ServicioTerminado	Servicio	Administración	US-08
IncidenciaDocumentada	Servicio	Administración	US-09
ExtraAceptado / ExtraPendienteDeConfirmacion / ExtraConfirmado	Servicio	Coordinador, Administración	US-10
AjustePropuesto / AjusteAceptadoPorCliente	Servicio	Cliente, Administración	US-11
AccesoNegadoRegistrado / ServicioNoRealizado	Servicio	Administración	US-13
ReprogramacionSolicitada / ReprogramacionAprobada / ReprogramacionRechazada	Servicio	Administración, Cliente	US-14; US-15
ServicioModificado / ServicioCancelado	Servicio	Administración (facturación)	US-16; BR-08
ServicioCerradoParaFacturacion	Servicio	Facturación (externo)	CU-05; US-17

Alcance en el código. El código base implementa 15 de estos eventos (de CotizacionEmitida a ServicioCerradoParaFacturacion). No implementa RecordatorioDeConfirmacionGenerado, AjustePropuesto, AjusteAceptadoPorCliente, AccesoNegadoRegistrado, ServicioNoRealizado, los de reprogramación ni ServicioModificado y ServicioCancelado, porque las historias que los originan quedan fuera de CU-01 a CU-05.

Fichas de Aggregates

Cotización

- **Raíz:** Cotización (Folio). Contiene DatosDelInmueble, Frecuencia, extras cotizados, Estimacion y Estado.
- **Responsabilidades:** emitirse (con folio y estado Vigente), aceptarse y rechazarse.
- **Invariantes:**
    - **C1.** No se emite sin los cinco datos: ubicación, superficie, tipo de inmueble, frecuencia y extras (US-01).
    - **C2.** Al emitirse recibe un folio único y queda Vigente (US-01).
    - **C3.** La estimación considera superficie, tipo, frecuencia y extras, y el coordinador puede editarla antes de emitir (US-02). Una vez emitida ya no se edita: US-02 solo permite editarla antes de emitir.
    - **C4.** Solo una cotización Vigente puede aceptarse o rechazarse. Aceptada y Rechazada son estados finales (decisión: la U2 no define transiciones de salida) (glosario; US-03).
    - **C5.** Aceptar no asigna cuadrilla definitiva (BR-01; US-03).
- **Eventos:** CotizacionEmitida, CotizacionAceptada, CotizacionRechazada. **Referencias:** ClienteId.

Servicio

- **Raíz:** Servicio (Folio). Contiene RegistroDeTiempo, AsignacionDePersonal, Incidencias, Extras, AjusteDePrecioYTiempo, SolicitudDeReprogramacion y el historial de cambios (RNF-03; US-19).
- **Estados:** Programado (tentativa), CuadrillaConfirmada, EnEjecucion, Terminado, CerradoParaFacturacion, EnRiesgo, NoRealizado y Cancelado.
- **Responsabilidades:** confirmar cuadrilla, reportar ausencia y reasignar, iniciar y terminar, documentar incidencias, registrar y confirmar extras, proponer y resolver ajustes, registrar acceso negado, solicitar y resolver reprogramaciones, cancelar y cerrar para facturación. Ajuste (S7), material especial (S8), acceso negado (S9), reprogramación (S10) y cancelación (S11) son diseño: no están en el código base.
- **Invariantes:**
    - **S1.** Solo nace de una Cotización Aceptada, uno por cotización, vinculado a su Folio, en estado Programado y sin cuadrilla definitiva (BR-01; US-03).
    - **S2.** Pasa a CuadrillaConfirmada solo si el coordinador valida la disponibilidad real; mientras no lo haga, la cuadrilla no se considera confirmada (BR-02; US-04).
    - **S3.** El término no puede registrarse sin inicio ni ser anterior a él (US-08).
    - **S4.** Una ausencia crea una Incidencia vinculada al Folio y notifica a Administración. La reasignación conserva al empleado original y al sustituto. Sin reemplazo disponible, el Servicio pasa a EnRiesgo y se sugiere contactar al cliente (BR-03; US-06; US-07).

  - **S5.** Toda incidencia lleva tipo, descripción y hora, y nace Pendiente de revisión (US-09).
  - **S6.** Un extra del catálogo se acepta directo; uno fuera del catálogo queda pendiente de confirmación del coordinador (BR-04; US-10).
  - **S7.** Un ajuste por inmueble mayor lo recalcula el coordinador, requiere aceptación explícita del cliente antes de continuar y se registra con su motivo (BR-05; US-11).
  - **S8.** El material especial se registra como incidencia con la decisión tomada (continuar parcial o reprogramar) y no asigna quién provee el material (US-12).
  - **S9.** El acceso negado solo se registra después del tiempo de cortesía configurado y deja el Servicio NoRealizado, con su incidencia (US-13).
  - **S10.** Una solicitud de reprogramación pendiente no modifica la fecha original. Solo Administración la aprueba o rechaza, y conserva el motivo (BR-06; US-14; US-15).
  - **S11.** Solo se cancela un servicio aún no iniciado, por el coordinador, y Administración queda notificada (BR-08; US-16).
  - **S12.** Se cierra para facturación solo con término registrado y sin incidencias ni extras Pendientes; si los hay, se bloquea y se indica cuál falta. Solo Administración lo ejecuta (BR-07; RNF-02; US-17).
  - **S13.** Toda modificación relevante (reasignación, ajuste, cancelación) conserva quién, cuándo y por qué (RNF-03).
- **Referencias:** Folio de la cotización, CuadrillaId, EmpleadoId (en AsignacionDePersonal).

Cuadrilla y Empleado

- **Cuadrilla:** raíz mínima con CuadrillaId e integrantes (EmpleadoId). No guarda agenda: la disponibilidad de una fecha se deriva de los servicios ya confirmados, como pide US-05 ("la información incluye esos servicios confirmados"). Así una confirmación modifica un solo Aggregate.
- **Invariantes:**
  - **Q1.** Una cuadrilla no se confirma en dos servicios el mismo día (supuesto: US-05 la muestra como libre u ocupada por día; validar). La verifica AsignadorDeCuadrilla y se protege dentro de la transacción.
- **Empleado:** raíz mínima con EmpleadoId y disponibilidad para cubrir ausencias. No se modela nómina (fuera de alcance).

ConfiguracionDePoliticas (soporte)

- **Contiene:** tiempo de cortesía, catálogo de extras, criterio de precio significativo, política de cobro por acceso negado y penalización por reprogramación tardía. **Ninguno tiene valor asignado** hasta validarse con el negocio.
- **Invariante:** solo Administración la modifica y cada cambio conserva quién, cuándo y por qué (RNF-02; RNF-03; RNF-04).
- **Uso:** los Domain Services y los métodos del Servicio la reciben como parámetro; las entidades no la consultan por su cuenta.

Decisiones dudosas o descartadas

Elemento	Decisión	Razón
Aggregate Factura y contexto Facturación	Descartado	El BRD excluye la integración contable; el sistema solo cierra para facturación (H-03)
Aggregate Cliente	Descartado (solo ClienteId)	La U2 no define comportamiento del cliente en el dominio
Descuentos, cargos porcentuales, límite de 8 cuadrillas, rangos de m²	Descartados	Valores y reglas inventados; la U2 no los define (H-06)
Estado Suspendida	Descartado	No existe en la U2; BR-08 solo habla de modificar o cancelar
Solicitud de cotización como estado previo a Vigente	Se modela como registro de entrada, no como estado	El glosario solo define Vigente, Aceptada y Rechazada
Agenda dentro de Cuadrilla	Descartado	La disponibilidad se deriva de servicios confirmados; evita una transacción de dos Aggregates
Servicio y Cotización en un solo Aggregate	Descartado	Tienen ciclos de vida distintos (la cotización se cierra al aceptarse; el servicio continúa meses o semanas después)

Hallazgos y pendientes sobre la Unidad 2

Contradicciones y datos faltantes de la U2 detectados al diseñar, según la regla de continuidad de la actividad. Cada uno se resolvió con la decisión más lógica según la U2 y sin inventar valores; la decisión y su reflejo en el diseño se indican en la tabla.

Hallazgo	Origen en la U2	Decisión tomada y dónde se refleja
H-01 · Un folio o dos	US-03 habla del folio de la cotización; US-06 y US-08, del folio del servicio.	Decisión: un solo Folio, asignado a la cotización y conservado por el Servicio, porque US-06 y US-08 lo usan para vincular el servicio y US-03 vincula la programación a la cotización. Glosario: Folio; ADR-002; invariante S1.

Hallazgo	Origen en la U2	Decisión tomada y dónde se refleja
H-02 · Qué es «precio significativo»	El BRD y el glosario condicionan la confirmación de un extra a que modifique el precio de forma significativa; US-10 solo distingue por catálogo.	Se aplica el criterio de catálogo; el umbral queda sin valor en ConfiguracionDePoliticas. ADR-003; PoliticaDeExtras.
H-03 · Facturar o solo cerrar para facturación	El BPMN muestra la tarea «Facturar servicio» y el artefacto Factura; el BRD deja fuera la automatización de la facturación hacia sistemas contables.	Solo cierre: el evento ServicioCerradoParaFacturacion sale por FacturacionPort y no existe Aggregate Factura. ADR-004; glosario: Facturación.
H-04 · Empleado, Cuadrilla y capacidad	El glosario no define Empleado ni cómo se compone una Cuadrilla; el BRD no confirma cuántas cuadrillas hay ni su disponibilidad diaria.	Decisión: raíces mínimas con EmpleadoId y CuadrillaId, sin límite de cuadrillas, porque la U2 no da más datos. Ficha Cuadrilla y Empleado; glosario: Empleado.

Hallazgo	Origen en la U2	Decisión tomada y dónde se refleja
H-05 · Transiciones y estados sin definir	(a) Qué sigue si la cuadrilla propuesta no está disponible (el BPMN no deja clara la ruta). (b) Qué pasa si el cliente rechaza un ajuste o el coordinador rechaza un extra. (c) Si un ajuste sin respuesta bloquea el cierre. (d) Cómo sale una incidencia de «Pendiente de revisión». (e) El glosario solo define estados de la Cotización.	Decisiones: (a) si la cuadrilla no está disponible, el sistema rechaza la confirmación (CUADRILLA_NO_DISPONIBLE) y el coordinador propone otra, porque US-04 dice que sin validación no se considera confirmada. (b) Un ajuste rechazado por el cliente deja el servicio con las condiciones originales y un extra rechazado por el coordinador no se cobra; ambos conservan el motivo (RNF-03). (c) Un ajuste sin respuesta no bloquea el cierre: US-17 solo bloquea por incidencias y extras. (d) Administración marca Revisada la incidencia, porque es quien la consulta (US-09, US-07). (e) Estados documentados en el glosario actualizado. Invariantes S2 y S12.
H-06 · Valores y datos sin definir	La U2 no da tarifario, moneda, unidades de superficie, descuentos ni rangos; US-03 no indica de dónde sale la fecha de la programación tentativa; el BRD pide «evidencia» de incidencias y extras, pero US-09 solo pide tipo, descripción y hora.	No se inventan valores: CalculadoraDeEstimacion queda en diseño, Dinero sin moneda, la fecha entra como dato opcional (sin ella no se confirma cuadrilla) y la evidencia se limita a tipo, descripción y hora.

Hallazgo	Origen en la U2	Decisión tomada y dónde se refleja
H-07 · Reglas sin identificador y RNF repetido	El BRD no numera reglas ni requisitos no funcionales; su cuarto RNF (no facturable con pendientes) repite la regla de cierre; el glosario no define Rol ni Actor aunque RNF-02 exige permisos por rol.	El equipo asigna BR-01 a BR-08 y RNF-01 a RNF-04 en el orden del texto (tabla del Alcance); RNF-04 es el de valores configurables. Se agregan Rol y Actor al glosario. Matriz de trazabilidad.
H-08 · Historias dentro del alcance sin cobertura completa	US-14 a US-16 (reprogramación y cancelación) y BR-06 y BR-08 no aparecen en el BPMN; US-04 pide un recordatorio a 24 h sin actor ni paso en el BPMN; el panel y el historial (US-18, US-19) no tienen paso en el BPMN; US-11 y US-13 sí están, pero fuera de CU-01 a CU-05.	Decisión: se diseñan en el dominio (S7 a S11, eventos) y en las consultas, y no se programan porque quedan fuera de los cinco casos de uso. Alcance del código base; matriz de trazabilidad.

Glosario actualizado

La versión completa está en el entregable Glosario actualizado (23 términos de la U2 y 17 nuevos). Aquí se listan los términos nuevos o precisados.

Término	Definición	Cambio
Folio	Identificador único de la cotización, conservado por el Servicio que nace de ella	Nuevo (supuesto, H-01)
Servicio	Se mantiene la definición de la U2; en el diseño la cotización aceptada se referencia por el Folio (ADR-002)	Precisado
Estados de la cotización	Vigente, Aceptada y Rechazada	Nuevo término; mismos estados de la U2
Extra cotizado	Extra incluido en la cotización original	Nuevo
Extra (en sitio)	Extra / servicio adicional del glosario U2: solicitado en sitio y no cotizado	Precisado
Tipo de incidencia	Ausencia, Acceso negado, Material especial o Daño reportado	Nuevo
Estados de la incidencia	Pendiente de revisión (recién registrada) y Revisada (Administración ya la revisó)	Nuevo (US-09, H-05)

Término	Definición	Cambio
Estados del extra	Aceptado, Pendiente de confirmación y Confirmado	Nuevo (US-10)
Estados del Servicio	Programado, CuadrillaConfirmada, EnEjecucion, Terminado, CerradoParaFacturacion, EnRiesgo, NoRealizado, Cancelado	Nuevo
Política de negocio	Valores revisables por Administración, sin valor hasta validarse (RNF-04)	Nuevo
Rol y Actor	Rol: Cliente, Coordinador, Supervisor o Administración. Actor: quien ejecuta una operación, con su id y su rol	Nuevo (RNF-02, H-07)
Empleado	Persona del personal operativo que puede integrar una cuadrilla, ausentarse o cubrir una ausencia	Nuevo (H-04)
Cliente	Persona que solicita y acepta cotizaciones; en el sistema solo se referencia por su identificador	Nuevo
Datos del inmueble	Ubicación, superficie y tipo de inmueble sobre los que se cotiza	Nuevo
Frecuencia	Periodicidad con la que se contrata el servicio	Nuevo
Estimación	Duración, personal requerido y precio de una cotización; el coordinador puede editarla antes de emitirla (US-02)	Nuevo
Asignación de personal	Registro de una reasignación: empleado original, sustituto, quién, cuándo y por qué (US-07, RNF-03)	Nuevo
Disponibilidad de cuadrilla	Condición libre u ocupada en una fecha; no se registra aparte, se deduce de los servicios con cuadrilla confirmada (US-05)	Nuevo
Facturación	Proceso externo fuera del alcance; CleanPro solo cierra el servicio para facturación y comunica ese hecho (H-03)	Nuevo

Diseño de Service Layer

La capa de aplicación expone una operación por intención completa del usuario. Cada servicio recibe la solicitud, obtiene el Aggregate, invoca su comportamiento, confirma la Unit of Work y devuelve un resultado. Las reglas viven en el dominio; el servicio no calcula umbrales ni decide transiciones.

Servicio	Método o caso de uso	Entrada	Colaboradores	Salida	Reglas que delega
CotizacionesAppService	Aceptar cotización (CU-01)	AceptarCotizacion (folio, clienteId, fecha)	CotizacionRepository, ServicioRepository, UnitOfWork	ServicioProgramado	BR-01, US-03: Verifica que esté Vigente y crea el Servicio solo con programación tentativa (sin cuadrilla definitiva).
ProgramacionAppService	Confirmar cuadrilla el día previo (CU-02)	ConfirmarCuadrilla (folio, cuadrillaId, coordinadorId)	ServicioRepository, CuadrillaRepository, AsignadorDeCuadrilla, UnitOfWork	CuadrillaConfirmada o error de disponibilidad	BR-02, US-04: Confirma la cuadrilla solo si el coordinador valida su disponibilidad real en la fecha prevista.
PersonalAppService	Reportar ausencia y reasignar (CU-03)	ReportarAusencia, ReasignarPersonal	ServicioRepository, EmpleadoRepository, NotificacionPort, UnitOfWork	AusenciaReportada, PersonalReasignado, o ServicioEnRiesgo	BR-03, US-06, US-07: Crea la incidencia, conserva el historial del empleado original y del sustituto; si no hay reemplazo, el servicio pasa a EnRiesgo.
EjecucionAppService	Registrar extra en sitio y confirmar extra fuera de catálogo (CU-04)	RegistrarExtra (folio, descripcion, supervisorId); ConfirmarExtra (folio, extraId, coordinadorId)	ServicioRepository, PoliticaDeExtras, ConfiguracionRepository, UnitOfWork	ExtraAceptado, ExtraPendienteDeConfirmacion o ExtraConfirmado	BR-04, US-10: PoliticaDeExtras decide si el extra está en el catálogo (aceptación directa) o requiere al coordinador. Que solo el Coordinador confirme (RNF-02) lo valida Servicio.
CierreAppService	Cerrar para facturación (CU-05) y revisar incidencia	CerrarParaFacturacion (folio, adminId); RevisarIncidencia (folio, incidenciaId, adminId)	ServicioRepository, UnitOfWork, FacturacionPort	CerradoParaFacturacion, error CIERRE_BLOQUEADO o incidencia Revisada	BR-07, US-17: Servicio bloquea el cierre si existen incidencias o extras pendientes y señala cuál falta. revisarIncidencia: Administración marca Revisada una incidencia Pendiente de revisión (supuesto, H-05).

Revisión

¿El nombre del método expresa una acción del negocio?

Sí. Se utilizan verbos operativos alineados al glosario de CleanPro (aceptarCotizacion, confirmarCuadrilla, cerrarParaFacturacion). No existen servicios genéricos tipo "CRUD" o "AutorizacionesGenerales".

¿El método coordina o contiene decisiones que deberían estar en el dominio?

Solamente coordina. Por ejemplo, el bloqueo del cierre por incidencias pendientes lo evalúa directamente el Aggregate Servicio protegiendo sus propias invariantes, no el CierreAppService.

¿El servicio depende de interfaces o de detalles concretos de base de datos?

Depende exclusivamente de interfaces (Repositorios para obtener Aggregates, UnitOfWork para la persistencia, y puertos de notificación). Abstrae cualquier detalle de SQL.

¿La operación tiene un inicio y un final transaccional comprensible?

Sí. Cada operación inicia y termina en una sola Unit of Work que modifica un único Aggregate por transacción (con la excepción documentada y justificada de aceptarCotizacion, que acepta la Cotización y crea el Servicio, dos Aggregates en una misma transacción).

¿Los errores del dominio pueden explicarse con términos del negocio?

Sí. Las operaciones retornan conceptos propios de las reglas de negocio, como CIERRE_BLOQUEADO o CUADRILLA_NO_DISPONIBLE, en lugar de exponer excepciones técnicas del código.

Interfaces de Repository

Los repositorios se definen en la capa de dominio y abstraen los detalles de la base de datos. Sus métodos expresan únicamente lo que el negocio necesita guardar o recuperar manejando los Aggregates completos. El lado de lectura tiene su propia interfaz en la capa de aplicación (DisponibilidadReadRepository, ver CQRS).

Repository	Necesidad del dominio	Métodos propuestos	Objeto que recupera o guarda
CotizacionRepository	Persistir la creación de una cotización o recuperar una cotización previa para procesar su aceptación o rechazo.	guardar(Cotizacion), obtenerPorFolio(Folio)	Aggregate Cotizacion
ServicioRepository	Recuperar la entidad principal para gestionar todo su ciclo de vida operativo (confirmación, tiempos, incidencias y extras) y saber qué servicios tienen cuadrilla confirmada en una fecha (disponibilidad derivada, US-05).	guardar(Servicio), obtenerPorFolio(Folio), obtenerConfirmadosPorFecha(fecha)	Aggregate Servicio
CuadrillaRepository	Consultar y verificar la existencia e integrantes de una cuadrilla asignada antes de confirmarla.	obtenerPorId(CuadrillaId)	Aggregate mínima Cuadrilla
EmpleadoRepository	Recuperar al empleado ausente o al sustituto para reasignar personal y verificar que puede cubrir la ausencia (US-07).	obtenerPorId(EmpleadoId)	Aggregate mínimo Empleado
ConfiguracionRepository	Obtener los parámetros operativos vigentes (tiempo de cortesía, catálogo de extras) para que los servicios evalúen políticas sin acoplar valores fijos.	obtenerPoliticasVigentes()	Aggregate ConfiguracionDePoliticas

Implementación en Infraestructura

Las interfaces viven en el dominio y cada una tiene su propia implementación en infraestructura: ServicioRepositoryEnMemoria, CotizacionRepositoryEnMemoria, CuadrillaRepositoryEnMemoria, EmpleadoRepositoryEnMemoria, ConfiguracionRepositoryEnMemoria y, para lectura, DisponibilidadReadRepositoryEnMemoria, además de UnitOfWorkEnMemoria, NotificacionAdapter y FacturacionEventAdapter. Estas implementaciones en memoria son las que contiene el código base: entregan y guardan copias, de modo que un cambio solo persiste al llamar a guardar() y commit(). Para producción se propone un ORM (como Entity Framework o Hibernate) que implemente las mismas interfaces, traduzca los Aggregates a tablas (Incidencias y Extras en las suyas, devolviendo siempre el Servicio ensamblado) y pase las mismas pruebas de contrato

(tests/test_contrato_repositorios.py). Esa implementación SQL es una propuesta y no existe en el código. El dominio nunca conoce las tablas ni el ORM.

Definición del alcance transaccional (Unit of Work)

El patrón Unit of Work coordinará las transacciones para asegurar que las operaciones se completen juntas y el sistema no quede en un estado inconsistente.

- **Inicio de la transacción:** La transacción inicia exactamente cuando la solicitud entra a un método de la Service Layer (por ejemplo, al ejecutar EjecucionAppService.registrarExtra()).
- **Desarrollo y cambios que incluye:** El Application Service recupera el Aggregate a través del repositorio y ejecuta la lógica de dominio. Los cambios (como agregar el extra al Servicio) ocurren primero en memoria dentro del Aggregate; solo se guardan al llamar a guardar().
- **Fin y Commit:** Al finalizar el método exitosamente, el servicio llama a UnitOfWork.commit(), enviando todos los cambios del Aggregate a la base de datos de forma atómica.
- **Manejo de errores (Rollback):** Si el dominio detecta que se rompe una invariante (por ejemplo, Administración intenta cerrar para facturación un servicio con incidencias pendientes, CIERRE_BLOQUEADO), se lanza una excepción de dominio. El Application Service la intercepta, llama a UnitOfWork.rollback() para descartar cualquier modificación y devuelve el error en términos del negocio. No se guarda ningún dato parcial. Los eventos hacia puertos externos (FacturacionPort, NotificacionPort) se publican después del commit (ADR-004).

Separar Commands y Queries

Se aplica un CQRS básico para diferenciar las operaciones que cambian el estado y protegen invariantes de las que solo proyectan datos. Las consultas leen mediante interfaces de lectura propias (DisponibilidadReadRepository) y devuelven View Models sin cargar Aggregates; en producción pueden ir directo a la base de datos (Transaction Script) y en el código base se resuelven en memoria. Los 8 Commands del código (AceptarCotizacion, ConfirmarCuadrilla, ReportarAusencia, ReasignarPersonal, RegistrarExtra, ConfirmarExtra, RevisarIncidencia y CerrarParaFacturacion) están implementados; la tabla detalla dos de ellos y las dos consultas.

Tipo	Nombre	Datos de entrada	Handler y Dependencias	Resultado	Justificación
Command	ConfirmarCuadrilla	folio, cuadrillaId, coordinadorId	**Handler:** ProgramacionAppService **Dependencias:** ServicioRepository, CuadrillaRepository, AsignadorDeCuadrilla, UnitOfWork	CuadrillaConfirmada	Cambia el estado del servicio y asigna una cuadrilla. Requiere pasar por las reglas del Aggregate Servicio.
Command	RegistrarExtra	folio, descripcion, supervisorId	**Handler:** EjecucionAppService **Dependencias:** ServicioRepository, PoliticaDeExtras, UnitOfWork	ExtraAceptado o Pendiente	Modifica el Aggregate Servicio agregando un servicio adicional y evaluando la política del catálogo.
Query	ConsultarDisponibilidad	fecha	**Handler:** ConsultasAppService **Dependencias:** DisponibilidadReadRepository	DisponibilidadDTO[] (cuadrilla_id, libre)	Solo lee datos de cuadrillas y servicios confirmados (US-05). No necesita proteger ninguna regla de negocio.
Query	ConsultarPanelOperativo	rangoDeFechas	**Handler:** ConsultasAppService (propuesta)	PanelDTO	Consolida información de incidencias, extras y servicios (US-18) para visualización. Instanciar Aggregates aquí sería muy

			Dependencias: PanelReadRepository (propuesta)		costoso. Es una propuesta: no está en el código base ni en el diagrama de clases, porque US-18 queda fuera de CU-01 a CU-05.

Diseño de DTOs y Modelos de Lectura

Para las operaciones de consulta (Queries), se diseñaron View Models específicos (DisponibilidadDTO, implementado, y PanelDTO, propuesto) que estructuran únicamente los datos requeridos por la interfaz de usuario (por ejemplo, el estado libre/ocupado de una cuadrilla o el resumen de incidencias abiertas). Estos objetos no reutilizan los modelos de escritura (Cotizacion o Servicio), evitando acoplar las estructuras de persistencia con las vistas de presentación.

Justificación de partes que no necesitan esta separación

El resto de las operaciones internas del dominio que no son de cara al usuario final o que forman parte directa del ciclo atómico de escritura (como la validación interna de políticas o el cálculo de estimaciones mediante CalculadoraDeEstimacion, que es diseño) no requieren una bifurcación de modelos, ya que su propósito exclusivo es mutar el estado y proteger las invariantes dentro del mismo flujo transaccional.

Revisar la propuesta con SOLID

Se evaluó la arquitectura propuesta frente a los principios SOLID. Se registran seis hallazgos (SOL-1 a SOL-6), cada uno con evidencia, impacto y cambio realizado. Dos produjeron cambios nuevos en el diagrama o en el código (SOL-2 y SOL-4), dos verificaron correcciones hechas antes de la revisión (SOL-1 y SOL-6) y dos confirmaron decisiones (SOL-3 y SOL-5). Los riesgos que no se corrigieron quedan explícitos.

Hallazgo SOL-1: Responsabilidad Única (SRP)

- **Evidencia:** El Aggregate Servicio concentra 13 invariantes (S1 a S13) y 8 estados; los Application Services solo coordinan (por ejemplo, CierreAppService no evalúa BR-07, lo hace Servicio.cerrarParaFacturacion). En la primera versión los permisos por rol y el criterio de «precio significativo» tentaban a vivir en la capa de aplicación.
- **Impacto:** Servicio es un Aggregate grande: cada política nueva podría modificarlo. Dividirlo rompería S12 (BR-07), por eso no se hizo.
- **Cambio realizado:** Se verificó que los servicios de aplicación reciben un Actor con Rol y que el dominio valida el permiso (RNF-02), y que el umbral de precio está en ConfiguracionDePoliticas; ambas correcciones se tomaron en la iteración 3 de la bitácora y no reaparecieron. El Aggregate no se divide y queda como riesgo a vigilar.

Hallazgo SOL-2: SRP e ISP en la persistencia

- **Evidencia:** La primera versión del diagrama tenía una sola caja RepositoriosSql que implementaba las cinco interfaces de Repository.
- **Impacto:** Una clase con varias razones para cambiar y con métodos que algunos clientes no usan.
- **Cambio realizado:** Una implementación por Repository (ServicioRepositoryEnMemoria, CotizacionRepositoryEnMemoria, etc.), tanto en el diagrama de clases como en el código.

Hallazgo SOL-3: Abierto/Cerrado (OCP)

- **Evidencia:** Las reglas volátiles (tiempo de cortesía, catálogo de extras) viven en ConfiguracionDePoliticas y PoliticaDeExtras la recibe por parámetro.
- **Impacto:** Si el negocio cambia el catálogo o la política de cobro por acceso negado (RNF-04), no se modifica el Aggregate Servicio. El riesgo aparece si las reglas cambian de forma por tipo de extra: habría que pasar a estrategias.
- **Cambio realizado:** Se confirma la decisión; hoy PoliticaDeExtras solo aplica el criterio de catálogo.

Hallazgo SOL-4: Sustitución de Liskov (LSP)

- **Evidencia:** La primera redacción hablaba del uso de interfaces, que es inversión de dependencias. El criterio de sustitución es otro: cualquier implementación de un Repository o de la Unit of Work debe devolver Aggregates completos, informar un folio inexistente con el mismo error de negocio y comportarse igual ante commit y rollback.
- **Impacto:** Una implementación SQL futura que no respetara ese contrato rompería a la capa de aplicación sin que cambiara una sola línea suya.
- **Cambio realizado:** Se reformuló el hallazgo y se agregó tests/test_contrato_repositorios.py: una batería de contrato (6 pruebas) para ServicioRepository, CotizacionRepository y UnitOfWork. Hoy se ejecuta sobre las versiones en memoria; la implementación SQL propuesta deberá heredarla y pasarla.

Hallazgo SOL-5: Segregación de Interfaces (ISP)

- **Evidencia:** La consulta de disponibilidad (US-05) no necesita guardar(): ConsultasAppService depende solo de DisponibilidadReadRepository, que expone disponibilidadPorFecha. En cambio, ServicioRepository incluye obtenerConfirmadosPorFecha, que solo usa AsignadorDeCuadrilla.
- **Impacto:** Los consumidores de lectura no dependen de operaciones de escritura. Si ServicioRepository sigue acumulando consultas, deberá separarse una interfaz de consulta.
- **Cambio realizado:** Se confirma la decisión de tener ConsultasAppService y DisponibilidadReadRepository independientes; obtenerConfirmadosPorFecha se mantiene por ahora.

Hallazgo SOL-6: Inversión de Dependencias (DIP)

- **Evidencia:** El dominio y la aplicación dependen de abstracciones (UnitOfWork, NotificacionPort, FacturacionPort). Dos puntos siguen siendo compromisos: AsignadorDeCuadrilla (dominio) depende de ServicioRepository y aceptarCotizacion toca dos Aggregates.
- **Impacto:** CleanPro no queda acoplado al sistema contable ni al proveedor de notificaciones. AsignadorDeCuadrilla depende de una abstracción, no de SQL, pero acopla el dominio a la persistencia; la excepción de dos Aggregates rompe la regla de uno por transacción.
- **Cambio realizado:** Se verificó que el concepto de Factura ya estaba fuera del dominio, reemplazado por el evento ServicioCerradoParaFacturacion (H-03, ADR-004; decisión de la iteración 2). Los dos compromisos quedan documentados (ADR-002); alternativas: pasar los servicios confirmados como dato y crear el Servicio al reaccionar a CotizacionAceptada.

Architecture Decision Records

Cada ADR documenta una decisión con consecuencias para el diseño de CleanPro. El ADR-001 corresponde a la selección del patrón de lógica de negocio; los ADR-002 a ADR-004 son decisiones adicionales tomadas durante la actividad. Los IDs de trazabilidad (BR, RNF, US, CU) vienen de la Unidad 2 y de la Etapa 1.

ID	Decisión	Estado
ADR-001	Domain Model como patrón de lógica de negocio, con Transaction Script solo para lecturas	Aceptada
ADR-002	Cotización y Servicio como Aggregates separados; la disponibilidad se deriva de los servicios confirmados	Aceptada
ADR-003	Políticas pendientes de validar como ConfiguracionDePoliticas, pasadas como parámetro	Aceptada
ADR-004	Facturación fuera del dominio: el cierre emite un evento mediante FacturacionPort	Aceptada

ADR-001 · Domain Model como patrón de lógica de negocio, con Transaction Script solo para lecturas

Campo	Contenido
Título	Domain Model como patrón de lógica de negocio, con Transaction Script solo para lecturas
Estado	Aceptada
Contexto	CleanPro necesita proteger invariantes que cruzan Servicio, Incidencia y Extra: no cerrar para facturación con pendientes (BR-07, US-17), no confirmar cuadrilla sin validar disponibilidad (BR-02, US-04), no registrar término antes del inicio (US-08) y no continuar un ajuste sin aceptación del cliente (BR-05, US-11). El BRD identifica como problemas los errores de facturación y la poca evidencia de incidencias y extras, y advierte que cada supervisor podría aplicar las políticas de forma distinta. El sistema no es grande (19 historias, 5 actores); lo complejo son los estados y las invariantes.
Alternativas	• Transaction Script: Simple y rápido. Repite la verificación de BR-07 en cada procedimiento que decida si un servicio es facturable y traslada al código el riesgo de políticas aplicadas de forma distinta. • Table Module: Encaja con datos tabulares y reportes, pero las reglas quedan repartidas por tabla y BR-03 y BR-07 cruzan tres. Haría falta otra capa coordinadora, que terminaría siendo Transaction Script con otro nombre. • Domain Model: Cada invariante vive en un solo lugar (el Aggregate Servicio) y usa el lenguaje del glosario. Cuesta más modelar y exige mapeo a persistencia.
Decisión	Domain Model para el flujo que cambia el estado de un Servicio (CU-01 a CU-05 y el resto del ciclo de vida). Transaction Script solo para consultas que no cambian estado ni protegen invariantes: disponibilidad (US-05), panel operativo (US-18) e historial (US-19). Table Module queda descartado. La razón no es el tamaño del sistema, sino que las invariantes de cierre, confirmación y ajuste deben tener un único dueño.
Consecuencias	Beneficios: BR-07 y las demás invariantes se aplican en un solo lugar y ninguna pantalla puede saltárselas; el código reutiliza el glosario. Costos: más esfuerzo inicial de modelado y una capa de mapeo a persistencia. Riesgos: sobre-modelar (convertir cada

	tabla en entidad). Trabajo posterior: acotar el Domain Model al flujo central y validar las políticas pendientes con el negocio.
Trazabilidad	BR-02, BR-05, BR-07; RNF-03, RNF-04; US-04, US-08, US-11, US-17; CU-01 a CU-05; BPMN: «¿Pendientes por documentar?»

ADR-002 · Cotización y Servicio como Aggregates separados; la disponibilidad se deriva de los servicios confirmados

Campo	Contenido
Título	Cotización y Servicio como Aggregates separados; la disponibilidad se deriva de los servicios confirmados
Estado	Aceptada
Contexto	La cotización termina su ciclo al ser aceptada o rechazada (Vigente, Aceptada, Rechazada), mientras que el Servicio continúa después, desde la programación tentativa hasta el cierre para facturación (US-19). Además, US-05 pide mostrar la disponibilidad por cuadrilla a partir de los servicios ya confirmados. Los pendientes de la U2 dejan abierto si el folio es uno solo o dos (H-01; US-03 habla del folio de la cotización; US-06 y US-08, del servicio).
Alternativas	• Un solo Aggregate Cotización-Servicio: Garantiza consistencia inmediata, pero mezcla dos ciclos de vida distintos y haría crecer el Aggregate sin límite. • Cuadrilla con agenda propia: La confirmación modificaría dos Aggregates (Servicio y Cuadrilla) en la misma transacción. • Aggregates separados vinculados por Folio; disponibilidad derivada: Cada transacción modifica un solo Aggregate; la consulta de disponibilidad sale de los servicios confirmados.
Decisión	Cotizacion y Servicio son Aggregates distintos que comparten el mismo Folio (supuesto pendiente de validar). Cuadrilla no guarda agenda: su disponibilidad en una fecha se calcula con AsignadorDeCuadrilla a partir de los servicios confirmados. Por eso ServicioRepository necesita el método obtenerConfirmadosPorFecha(fecha).
Consecuencias	Beneficios: transacciones pequeñas, un Aggregate por operación y coherencia con US-05. Costos: la comunicación entre Cotización y Servicio pasa por el evento CotizacionAceptada (con la excepción documentada de aceptarCotizacion, que acepta la Cotización y crea el Servicio en la misma transacción), y la regla de «una cuadrilla no se confirma en dos servicios el mismo día» (Q1) debe protegerse dentro de la transacción, porque es un supuesto sin validar. Trabajo posterior: confirmar con el negocio si el folio es único y si Q1 es correcta.
Trazabilidad	BR-01, BR-02; US-03, US-04, US-05, US-19; CU-01, CU-02; invariantes S1, S2 y Q1

ADR-003 · Políticas pendientes de validar como ConfiguracionDePoliticas, pasadas como parámetro

Campo	Contenido
Título	Políticas pendientes de validar como ConfiguracionDePoliticas, pasadas como parámetro
Estado	Aceptada
Contexto	El BRD deja explícitamente pendientes el tiempo de cortesía, el cobro por acceso negado, la penalización por reprogramación tardía, el criterio de precio «significativo»

	y el catálogo de extras, y exige que queden configurables (RNF-04). El riesgo documentado es que cada supervisor las aplique de forma distinta. Ninguno de estos valores existe en la U2, y no deben inventarse.
Alternativas	• Valores fijos en el código: Rápido, pero obliga a editar el Aggregate cada vez que el negocio valide un valor y obliga a inventar cifras. • Condicionales por política dentro de Servicio: Cada política nueva modifica el Aggregate (viola OCP). • Aggregate de soporte ConfiguracionDePoliticas versionado: Los valores se cambian sin tocar el dominio; Administración los modifica y queda registro de quién, cuándo y por qué.
Decisión	Crear el Aggregate de soporte ConfiguracionDePoliticas, modificable solo por Administración y versionado. Las entidades no lo consultan por su cuenta: los Domain Services (PoliticaDeExtras) y los métodos de Servicio lo reciben como parámetro, obtenido por ConfiguracionRepository.obtenerPoliticasVigentes().
Consecuencias	Beneficios: se pueden aplicar las políticas sin inventar valores y el cambio de una política no obliga a modificar Servicio. Costos: una dependencia extra en los servicios de aplicación que las usan (EjecucionAppService) y la necesidad de definir qué ocurre mientras una política no tenga valor. Trabajo posterior: taller de validación con el negocio para asignar los valores.
Trazabilidad	BR-04; RNF-02, RNF-03, RNF-04; US-10, US-13, US-15; BRD: Restricciones y supuestos; CU-04

ADR-004 · Facturación fuera del dominio: el cierre emite un evento mediante FacturacionPort

Campo	Contenido
Título	Facturación fuera del dominio: el cierre emite un evento mediante FacturacionPort
Estado	Aceptada
Contexto	El BPMN muestra la tarea «Facturar servicio», pero el BRD deja fuera de alcance la integración con sistemas contables externos. El sistema solo debe garantizar que un servicio no se cierre para facturación con incidencias o extras pendientes (BR-07, US-17).
Alternativas	• Modelar Factura como Aggregate: Contradice el alcance del BRD y obliga a inventar reglas contables que la U2 no define. • Llamar directamente al sistema contable desde CierreAppService: Acopla la aplicación a un sistema externo (viola DIP) y dificulta las pruebas. • Emitir el evento ServicioCerradoParaFacturacion mediante un puerto: El dominio solo declara el hecho; la infraestructura decide cómo comunicarlo.
Decisión	El Aggregate Servicio produce el evento ServicioCerradoParaFacturacion y CierreAppService lo publica a través de FacturacionPort después de confirmar la Unit of Work. No existe Aggregate Factura. FacturacionEventAdapter implementa el puerto en infraestructura.
Consecuencias	Beneficios: el dominio no depende del sistema contable y se puede probar con un adaptador falso. Costos: la entrega del evento ocurre fuera de la transacción, por lo que debe definirse qué pasa si la publicación falla. Trabajo posterior: definir un mecanismo de reintento si falla la publicación; el sistema solo cierra para facturación (H-03).

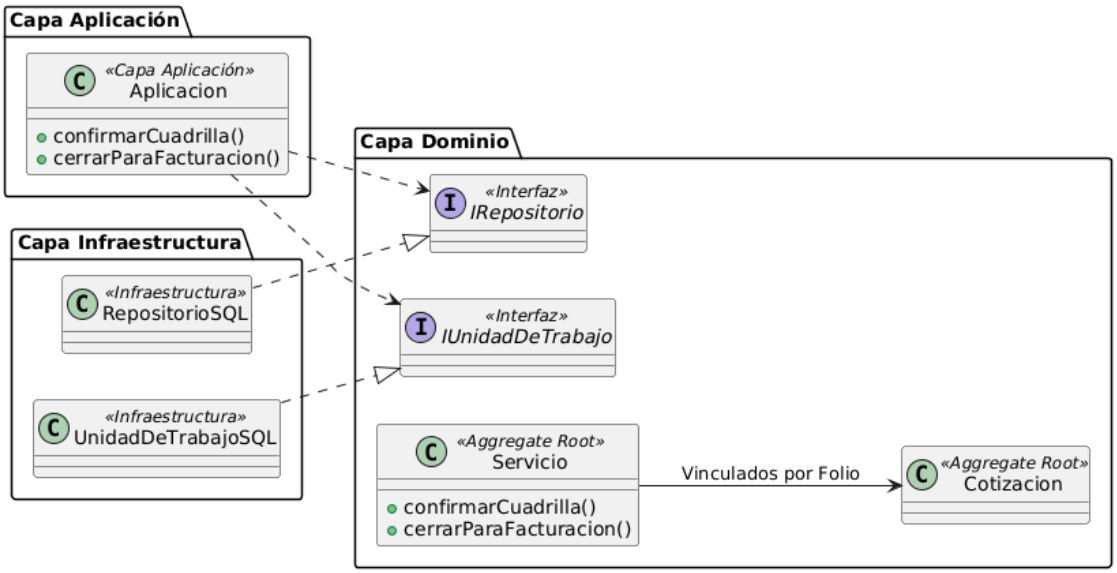
Trazabilidad	BR-07; RNF-02; US-17; CU-05; BRD: alcance excluido; H-03; BPMN: «Facturar servicio»

Repositorio

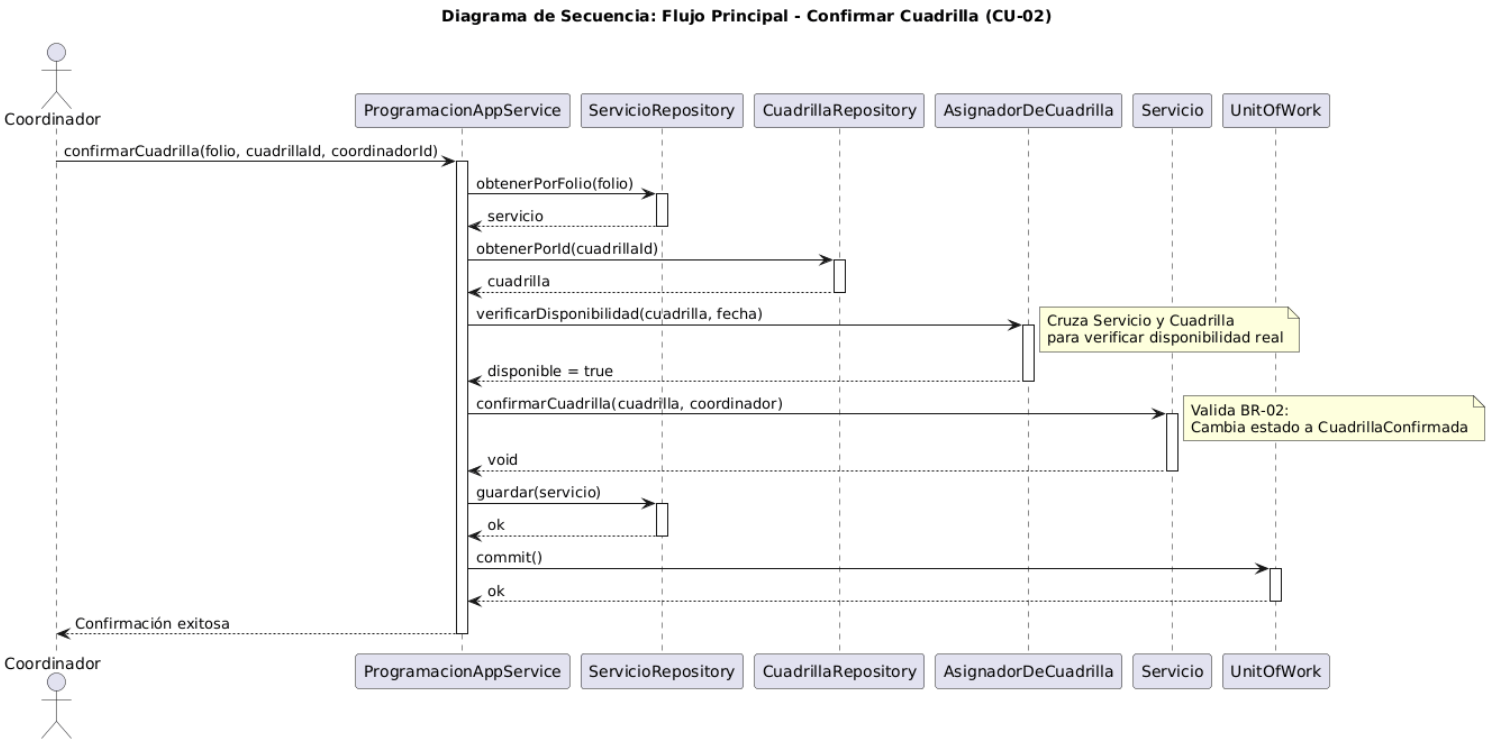
El código base, los diagramas, la documentación y las pruebas están en el repositorio de GitHub del equipo: **https://github.com/dani101909/cleanpro**

Las instrucciones de revisión están en el README (carpetas docs, diagrams, src y tests).

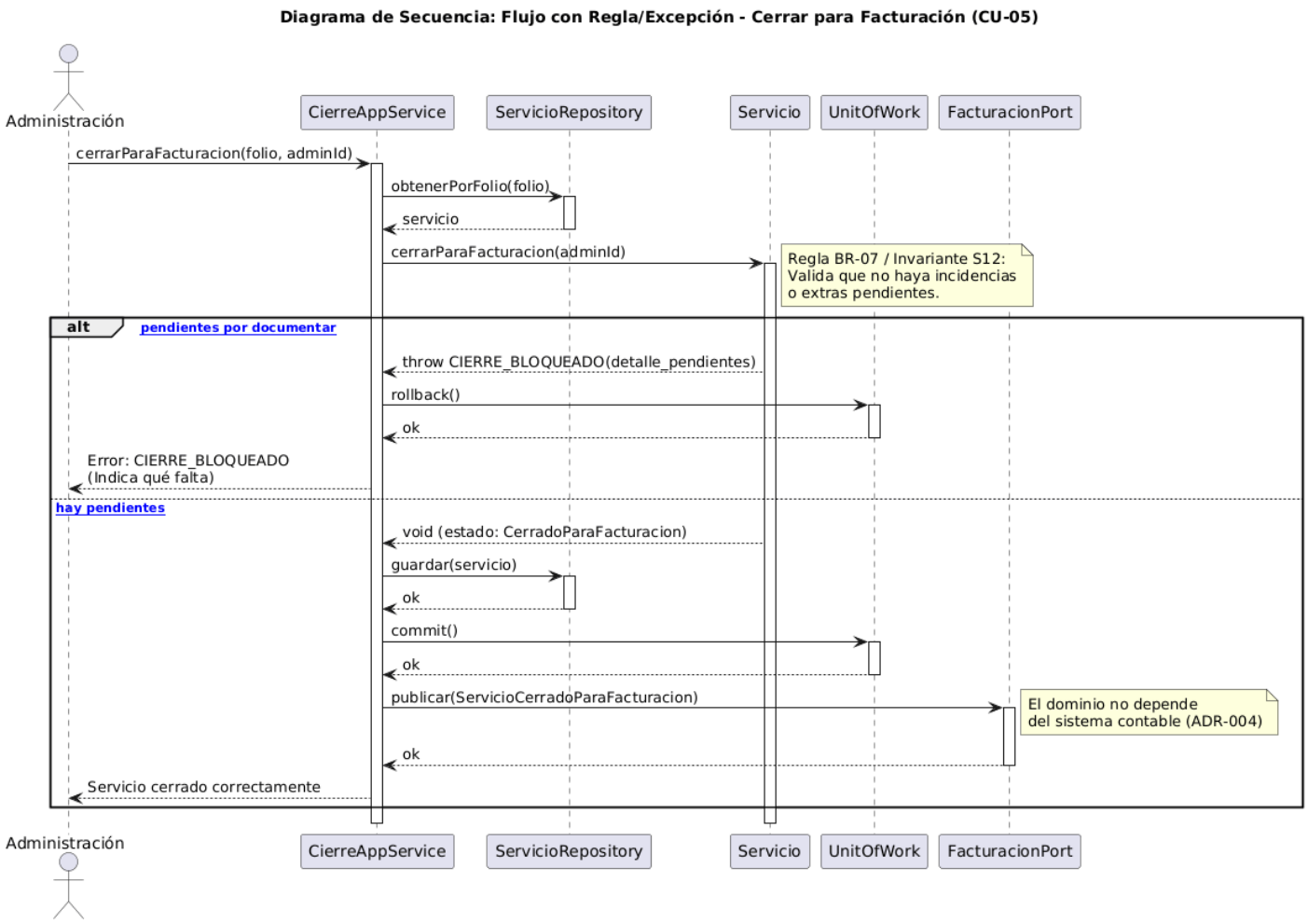
Diagrama de clases



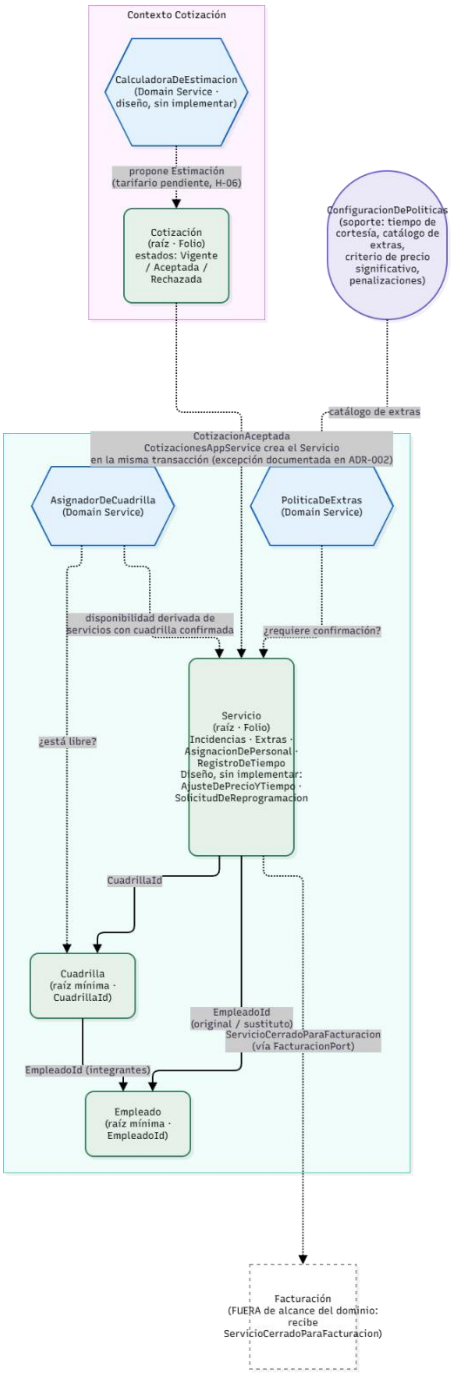
Secuencia de flujo principal



Secuencia de flujo con una regla o excepción



Mapa de Dominio



Conclusiones

El diseño convierte el análisis de la Unidad 2 en una arquitectura que separa dominio, aplicación e infraestructura. Cada decisión se apoya en una regla, historia o escenario del caso, y la trazabilidad está en la matriz.

Se eligió Domain Model para el flujo que cambia el estado de un servicio, y Transaction Script solo para consultas. La razón no fue el tamaño del sistema (19 historias y 5 actores), sino los estados y las invariantes que cruzan Servicio, Incidencia y Extra. La regla que más pesó fue BR-07: un servicio no se cierra para facturación con incidencias o extras pendientes. Atiende directamente dos problemas del BRD, los errores de facturación y la poca evidencia de incidencias y extras. Con este patrón la regla vive en un solo lugar, el Aggregate Servicio, y ninguna pantalla ni proceso puede saltársela.

Las reglas del negocio quedan en el dominio y los Application Services solo coordinan: obtienen el Aggregate, invocan su comportamiento, confirman la Unit of Work y devuelven un resultado en términos del negocio. Los Repository se definen con lenguaje del dominio y los detalles externos (notificaciones, facturación) quedan detrás de puertos. Las políticas aún no validadas se aíslan en ConfiguracionDePoliticas, de modo que cuando el negocio las defina no habrá que modificar el Aggregate Servicio.

La revisión con SOLID dejó seis hallazgos. Dos produjeron cambios (una implementación por Repository y pruebas de contrato), dos verificaron correcciones previas (permisos por rol en el dominio y Factura sustituida por un evento) y dos confirmaron decisiones (políticas configurables y separación de consultas). Quedan como riesgos a vigilar que el Aggregate Servicio concentra muchas invariantes, que AsignadorDeCuadrilla consulta un repositorio y que aceptarCotizacion toca dos Aggregates.

Sobre la Unidad 2 se registraron ocho hallazgos (H-01 a H-08), entre contradicciones y datos faltantes. Ninguno se resolvió inventando reglas: cada uno quedó como supuesto, política configurable o diseño pendiente de validar con el negocio.

El código base reproduce la arquitectura y la verifica con 35 pruebas: por invariante, por caso de uso y de contrato de los repositorios. Eso permite revisar en ejecución que las reglas se cumplen y que los flujos de excepción devuelven el resultado esperado. Las historias fuera de CU-01 a CU-05 (ajuste, acceso negado, reprogramación, cancelación, panel e historial) se diseñaron pero no se programaron.

Andrés Daniel Pérez Ordaz – 2163200

n conclusión, con esta actividad aprendí a convertir el análisis de negocio de la Unidad 2 en una propuesta de arquitectura de software. Pude ver que diseñar no consiste solo en dibujar diagramas, sino en decidir con criterio dónde debe estar cada regla del sistema y justificar por qué.

También aprendí a comparar los patrones de lógica de negocio y a elegir el más adecuado según las necesidades reales del caso. En CleanPro, Domain Model resultó la mejor opción porque permite proteger las reglas importantes, como no cerrar un servicio para facturación con incidencias o extras pendientes, en un solo lugar.

Esta actividad me ayudó a entender la importancia de separar el dominio, la aplicación y la infraestructura, así como de usar Repository, Unit of Work y CQRS para mantener el sistema ordenado y fácil de cambiar. La revisión con los principios SOLID me permitió detectar problemas y corregirlos antes de cerrar el diseño.

Jonathan Esquivel Anaya – 2160886

En conclusión, el documento vincula los cinco casos de uso con las reglas de negocio, historias de usuario, BPMN y glosario de la Unidad 2, y justifica Domain Model para el flujo que cambia estado porque protege invariantes entre Cotización, OrdenDeServicio, Incidencia y Extra, concentra reglas como BR-07 y centraliza políticas configurables; Transaction Script se limita a lecturas sin cambios de estado y Table Module se descarta por no garantizar bien esas invariantes. El mapa de dominio se acota a los contextos Cotización, Operación del servicio y Facturación, con ConfiguracionDePoliticas como soporte, usando lenguaje ubicuo, aggregates, value objects, domain services y domain events para expresar el negocio y proteger reglas clave. La Service Layer traduce intenciones de negocio en servicios de aplicación con contratos claros, reglas delegadas al dominio y cuidado de atomicidad, idempotencia y concurrencia, complementado con condiciones de seguridad, auditoría y observabilidad. Así, se acepta un mayor esfuerzo inicial de modelado y persistencia a cambio de reducir errores de cobro, evitar reglas dispersas y mejorar la evidencia de incidencias y extras, quedando pendientes validaciones con el stakeholder pero con una base acotada para continuar hacia la implementación.

Reporte

Patricia Lizeth Ramos Ortiz - 2161739

Esta actividad me mostró que diseñar una arquitectura consiste sobre todo en decidir dónde vive cada regla. Al principio pensé que Domain Model era la opción más completa por defecto, pero comparar las tres alternativas con el caso de CleanPro me obligó a justificarla con evidencia: no por el tamaño del sistema, sino porque reglas como no cerrar un servicio para facturación con incidencias o extras pendientes cruzan varios conceptos y necesitan un solo dueño. Ver esa regla concreta atacar los errores de facturación del negocio hizo que el patrón dejara de ser teoría.

También aprendí a desconfiar de mis primeras versiones. La Service Layer tendía a absorber decisiones, el diagrama tenía casi el doble de clases de las necesarias y la IA proponía elementos sin evidencia, como Factura o el estado Suspendida. Revisar cada propuesta contra el BRD, las historias y el glosario, y registrar por qué la aceptábamos o la corregíamos, fue más útil que la propuesta en sí. Los hallazgos sobre la Unidad 2 me enseñaron que declarar un supuesto pendiente es mejor que inventar un valor.

Me llevo la idea de que una buena arquitectura se puede defender con razones del negocio, y de que la IA es un buen copiloto solo si yo sigo siendo quien valida. Si tuviera que repetirlo, empezaría antes por validar con el negocio las políticas pendientes, porque son lo que más puede cambiar el diseño.